

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”
Отчет по рубежному контролю №2
Вариант Г13

Выполнил:

Студент группы ИУ5-35Б

Ким А. А.

Проверил:

Гапанюк Ю. Е.

подпись, дата

_____/_____

Москва 2025

Задание

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Листинг программы

main.py:

```
from library_system import Book, Library, BookLibrary, LibrarySystem
```

```
libraries = [  
    Library(1, 'Центральная библиотека'),  
    Library(2, 'Академическая библиотека'),  
    Library(3, 'Детская библиотека'),  
    Library(11, 'Арт-библиотека'),  
    Library(22, 'Абонемент научной литературы'),  
    Library(33, 'Английская библиотека'),  
]
```

```
books = [  
    Book(1, 'Война и мир', 1225, 1),  
    Book(2, 'Преступление и наказание', 671, 2),  
    Book(3, 'Мастер и Маргарита', 480, 3),  
    Book(4, '1984', 328, 3),  
    Book(5, 'Гарри Поттер', 500, 3),  
]
```

```
books_libraries = [  
    BookLibrary(1, 1),  
    BookLibrary(2, 2),  
    BookLibrary(3, 3),  
    BookLibrary(3, 4),  
    BookLibrary(3, 5),  
    BookLibrary(11, 1),  
    BookLibrary(22, 2),  
    BookLibrary(33, 3),  
    BookLibrary(33, 4),  
    BookLibrary(33, 5),  
]
```

```
def main():  
    system = LibrarySystem(libraries, books, books_libraries)
```

```

print("Задание Г1")
print('Список всех библиотек, у которых название начинается с буквы "А", и список книг в них:')

result1 = system.get_libraries_starting_with_a()
for lib, books_list in result1.items():
    print(f'{lib}: {books_list}')

print("\nЗадание Г2")
print(
    'Список библиотек с максимальным количеством страниц книг в каждой библиотеке,
отсортированный по максимальному количеству страниц:')

result2 = system.get_libraries_with_max_pages()
for lib, max_pages in result2:
    print(f'{lib}: {max_pages} стр.')

print("\nЗадание Г3")
print('Список всех связанных книг и библиотек, отсортированный по библиотекам:')

result3 = system.get_all_connections_sorted()
for lib_name, books_list in result3.items():
    print(f'{lib_name}: {books_list}')

if __name__ == '__main__':
    main()

```

library_system.py:

```

from operator import itemgetter

```

```

class Book:
    def __init__(self, id, title, pages, library_id):
        self.id = id
        self.title = title
        self.pages = pages
        self.library_id = library_id

```

```

class Library:
    def __init__(self, id, name):
        self.id = id
        self.name = name

```

```

class BookLibrary:
    def __init__(self, library_id, book_id):
        self.library_id = library_id
        self.book_id = book_id

```

```

class LibrarySystem:

```

```

def __init__(self, libraries, books, books_libraries):
    self.libraries = libraries
    self.books = books
    self.books_libraries = books_libraries

    self.one_to_many = self._create_one_to_many()
    self.many_to_many = self._create_many_to_many()

def _create_one_to_many(self):
    return [(b.title, b.pages, lib.name)
            for lib in self.libraries
            for b in self.books
            if b.library_id == lib.id]

def _create_many_to_many(self):
    many_to_many_temp = [(lib.name, bl.library_id, bl.book_id)
                          for lib in self.libraries
                          for bl in self.books_libraries
                          if lib.id == bl.library_id]

    return [(b.title, b.pages, lib_name)
            for lib_name, lib_id, book_id in many_to_many_temp
            for b in self.books if b.id == book_id]

def get_libraries_starting_with_a(self):
    libs_with_a = list(filter(lambda i: i[2].startswith('A'), self.many_to_many))

    res_1 = {}
    for book_title, pages, lib_name in libs_with_a:
        if lib_name not in res_1:
            res_1[lib_name] = []
        if book_title not in res_1[lib_name]:
            res_1[lib_name].append(book_title)

    return dict(sorted(res_1.items()))

def get_libraries_with_max_pages(self):
    res_2_unsorted = []

    for lib in self.libraries:
        lib_books = list(filter(lambda i: i[2] == lib.name, self.many_to_many))

        if len(lib_books) > 0:
            lib_pages = [pages for _, pages, _ in lib_books]
            lib_pages_max = max(lib_pages)
            res_2_unsorted.append((lib.name, lib_pages_max))
        else:
            res_2_unsorted.append((lib.name, 0))

    return sorted(res_2_unsorted, key=itemgetter(1), reverse=True)

def get_all_connections_sorted(self):
    res_3 = sorted(self.many_to_many, key=itemgetter(2))

```

```

grouped_result = {}
for book_title, pages, lib_name in res_3:
    if lib_name not in grouped_result:
        grouped_result[lib_name] = []
    grouped_result[lib_name].append(f"{book_title} ({pages} стр.)")

return dict(sorted(grouped_result.items()))

```

test_library_system.py:

```

import unittest
from library_system import Book, Library, BookLibrary, LibrarySystem

class TestLibrarySystem(unittest.TestCase):

    def setUp(self):
        self.libraries = [
            Library(1, 'Центральная библиотека'),
            Library(2, 'Академическая библиотека'),
            Library(3, 'Детская библиотека'),
            Library(11, 'Арт-библиотека'),
        ]

        self.books = [
            Book(1, 'Война и мир', 1225, 1),
            Book(2, 'Преступление и наказание', 671, 2),
            Book(3, 'Мастер и Маргарита', 480, 3),
        ]

        self.books_libraries = [
            BookLibrary(1, 1),
            BookLibrary(2, 2),
            BookLibrary(3, 3),
            BookLibrary(11, 1),
        ]

        self.system = LibrarySystem(self.libraries, self.books, self.books_libraries)

    def test_get_libraries_starting_with_a(self):
        result = self.system.get_libraries_starting_with_a()

        self.assertIsInstance(result, dict)

        self.assertIn('Академическая библиотека', result)
        self.assertIn('Арт-библиотека', result)

        self.assertNotIn('Центральная библиотека', result)
        self.assertNotIn('Детская библиотека', result)

        self.assertEqual(len(result['Академическая библиотека']), 1)
        self.assertEqual(result['Академическая библиотека'][0], 'Преступление и наказание')

```

```

def test_get_libraries_with_max_pages(self):
    result = self.system.get_libraries_with_max_pages()

    self.assertIsInstance(result, list)
    self.assertTrue(all(isinstance(item, tuple) for item in result))

    self.assertEqual(len(result), len(self.libraries))

    for i in range(len(result) - 1):
        self.assertGreaterEqual(result[i][1], result[i + 1][1])

    self.assertEqual(result[0][0], 'Центральная библиотека')
    self.assertEqual(result[0][1], 1225)

def test_get_all_connections_sorted(self):
    result = self.system.get_all_connections_sorted()

    self.assertIsInstance(result, dict)

    library_names = [lib.name for lib in self.libraries]
    for lib_name in library_names:
        self.assertIn(lib_name, result)

    keys = list(result.keys())
    self.assertEqual(keys, sorted(keys))

    for lib_name, books_list in result.items():
        self.assertIsInstance(books_list, list)
        for book_str in books_list:
            self.assertIn('(', book_str)
            self.assertIn(' стр.', book_str)

def test_empty_system(self):
    empty_system = LibrarySystem([], [], [])

    result1 = empty_system.get_libraries_starting_with_a()
    self.assertEqual(result1, {})

    result2 = empty_system.get_libraries_with_max_pages()
    self.assertEqual(result2, [])

    result3 = empty_system.get_all_connections_sorted()
    self.assertEqual(result3, {})

if __name__ == '__main__':
    unittest.main()

```

Скриншоты работы программы

```
(.venv) PS C:\Users\User\PycharmProjects\Labs 2025-2026\rk\rk2> python main.py
Задание Г1
Список всех библиотек, у которых название начинается с буквы "А", и список книг в них:
Абонемент научной литературы: ['Преступление и наказание']
Академическая библиотека: ['Преступление и наказание']
Английская библиотека: ['Мастер и Маргарита', '1984', 'Гарри Поттер']
Арт-библиотека: ['Война и мир']

Задание Г2
Список библиотек с максимальным количеством страниц книг в каждой библиотеке, отсортированный по максимальному количеству страниц:
Центральная библиотека: 1225 стр.
Арт-библиотека: 1225 стр.
Академическая библиотека: 671 стр.
Абонемент научной литературы: 671 стр.
Детская библиотека: 500 стр.
Английская библиотека: 500 стр.

Задание Г3
Список всех связанных книг и библиотек, отсортированный по библиотекам:
Абонемент научной литературы: ['Преступление и наказание (671 стр.)']
Академическая библиотека: ['Преступление и наказание (671 стр.)']
Английская библиотека: ['Мастер и Маргарита (480 стр.)', '1984 (328 стр.)', 'Гарри Поттер (500 стр.)']
Арт-библиотека: ['Война и мир (1225 стр.)']
Детская библиотека: ['Мастер и Маргарита (480 стр.)', '1984 (328 стр.)', 'Гарри Поттер (500 стр.)']
Центральная библиотека: ['Война и мир (1225 стр.)']
(.venv) PS C:\Users\User\PycharmProjects\Labs 2025-2026\rk\rk2> python -m unittest test_library_system.py
....
-----
Ran 4 tests in 0.001s

OK
(.venv) PS C:\Users\User\PycharmProjects\Labs 2025-2026\rk\rk2> python -m unittest test_library_system.py -v
test_empty_system (test_library_system.TestLibrarySystem.test_empty_system)
Дополнительный тест: пустая система ... ok
test_get_all_connections_sorted (test_library_system.TestLibrarySystem.test_get_all_connections_sorted)
Тест для задания Г3: все связи, отсортированные по библиотекам ... ok
test_get_libraries_starting_with_a (test_library_system.TestLibrarySystem.test_get_libraries_starting_with_a)
Тест для задания Г1: библиотеки, начинающиеся с 'А' ... ok
test_get_libraries_with_max_pages (test_library_system.TestLibrarySystem.test_get_libraries_with_max_pages)
Тест для задания Г2: библиотеки с максимальным количеством страниц ... ok
-----
Ran 4 tests in 0.002s

OK
(.venv) PS C:\Users\User\PycharmProjects\Labs 2025-2026\rk\rk2>
```